# ASSIGNMENT 3 – TASK 2

**SPOKEN DIGIT SEQUENTIAL CLASSIFICATION USING PRE-COMPUTED MFCC SEQUENCES**

Recurrent Neural Networks, LSTM, GRU, Bidirectional Models and Temporal Attention

**Student Name:** Sushmita Halder, Jyoti, Rajeev Kumar

**Roll Number:** 2621MA08, 2621MA12, 2621MA01

**Group:** 08 **Course:** Large Language Models

**Submission Date:** 12/10/2026

## Abstract

This project addresses sequential classification of spoken Arabic digits (0–9) using pre-computed Mel-Frequency Cepstral Coefficient (MFCC) sequences. Each utterance is represented as a variable-length sequence of 13-dimensional feature vectors. The notebook implements Vanilla RNN, LSTM and GRU classifiers, compares hyperparameter configurations, and explores bidirectional recurrent processing and temporal attention pooling. Model selection is based on validation accuracy, while the held-out test set is used for final evaluation. In the advanced comparison, the Bi-GRU model with temporal attention achieved the highest reported test accuracy of 98.73%. In the separate baseline hyperparameter experiment, the best LSTM configuration achieved 96.64% test accuracy among the selected RNN, LSTM and GRU models.

## 1. Introduction

Spoken digit recognition is a sequence-classification problem: an input utterance consists of a time-ordered series of acoustic feature vectors, and the system must predict one digit label for the complete sequence. Recurrent neural networks are suitable for this task because their hidden states carry information across timesteps. LSTM and GRU cells add gating mechanisms intended to help learn temporal dependencies. Bidirectional models process sequences in both temporal directions, while attention pooling learns which timesteps should contribute more strongly to the final utterance representation.

## 2. Objectives

- Load the pre-computed MFCC sequences from the UCI Spoken Arabic Digit dataset.
- Represent each utterance as a variable-length sequence of 13 MFCC coefficients per timestep.
- Handle variable sequence lengths through padding, sequence lengths and batch collation.
- Train and compare Vanilla RNN, LSTM and GRU architectures across hyperparameter settings.
- Select models using validation accuracy and evaluate the selected configurations on the test set.
- Plot training loss and training error across epochs.
- Visualize MFCC coefficient heatmaps for representative spoken digit utterances.
- Explore bidirectional recurrent networks and temporal attention pooling as bonus improvements.
- Compare performance using accuracy, classification reports and confusion matrices.

## 3. Dataset and Representation

The notebook uses the UCI Spoken Arabic Digit dataset, with pre-computed MFCC features supplied as text files rather than raw audio. The files are Train_Arabic_Digit.txt and Test_Arabic_Digit.txt. The dataset contains utterances from multiple speakers and ten target classes corresponding to digits 0 through 9.

| Property | Description |
|---|---|
| **Input feature** | Mel-Frequency Cepstral Coefficients (MFCC) |
| **Feature dimension** | 13 coefficients per timestep |
| **Sequence length** | Variable; notebook batches include padded sequences |
| **Classes** | 10 spoken digits: 0–9 |
| **Training utterances loaded** | 6,600 (as reported by initial loading output) |
| **Test utterances** | 2,200 |
| **Speaker-disjoint split output** | 5,610 training sequences; 990 validation sequences; 2,200 test sequences |
| **Example batch tensor** | (batch, time, MFCC) = (64, 57, 13) |

The notebook output reports 66 unique speakers, with a speaker-disjoint split of 56 training speakers and 10 validation speakers. A later integrity check reports 5,610 training and 990 validation sequences. These exact counts are retained here as recorded in the notebook; the total training/validation count differs slightly from the initial loading count, so the final report should be interpreted with that notebook output in mind.

## 4. Data Preparation and Batch Handling

### 4.1 Loading and labels

The code loads the train and test text files, parses utterance feature sequences and digit labels, and stores each utterance as a sequence rather than flattening all timesteps into one vector. The target labels are integer classes from 0 to 9.

### 4.2 Speaker-disjoint validation

The notebook separates speakers between training and validation to reduce the risk that utterances from the same speaker appear in both subsets. This provides a more meaningful estimate of generalization to speakers not used for training. The test set remains separate for final evaluation.

### 4.3 Padding and sequence lengths

Because utterances have different numbers of frames, the batch collation function pads sequences to the longest sequence in each mini-batch and retains each sequence's original length. The example output shows a batch tensor with shape (64, 57, 13). Padding allows efficient mini-batch processing while sequence lengths help the recurrent model ignore padded frames where appropriate.

### 4.4 Standardization

The notebook includes standardization of MFCC features. Standardization places coefficients on comparable scales, which can make neural-network optimization more stable.

## 5. Model Architectures

### 5.1 Vanilla RNN

A Vanilla RNN updates a hidden state sequentially using the current input and previous hidden state. The final sequence representation is used to classify the utterance into one of ten digit classes. RNNs are simple, but may have difficulty learning long-range dependencies.

### 5.2 LSTM

Long Short-Term Memory networks use input, forget and output gates together with a cell state. These mechanisms help preserve useful information over time and can improve learning over variable-length sequences.

### 5.3 GRU

Gated Recurrent Units use update and reset gates to control information flow. GRUs often provide a useful balance between temporal modeling capability and model complexity.

### 5.4 Bidirectional recurrent models (bonus)

Bidirectional RNN, LSTM and GRU models process the sequence in forward and reverse directions. Their combined representation can use both preceding and following context when classifying an utterance.

### 5.5 Temporal attention pooling (bonus)

Temporal attention pooling assigns learned weights to the recurrent outputs at different timesteps and combines them into a weighted sequence representation. This allows the model to emphasize frames that are especially informative for identifying the spoken digit. The notebook also visualizes attention weights for a representative utterance of digit 5.

## 6. Training Procedure and Hyperparameter Selection

The notebook trains baseline recurrent models and evaluates multiple configurations. Hyperparameters include recurrent cell type, hidden size, number of recurrent layers, dropout and learning rate. The saved baseline configurations include 64- or 128-unit RNNs, 64-, 128- or 256-unit LSTMs, and 64-, 128- or 256-unit GRUs, with one or two layers and learning rate 0.001. The best configuration for each baseline architecture is selected by validation accuracy, and the test set is used only for final evaluation.

The convergence rule specified in the notebook is based on the absolute difference between training errors (or losses as implemented in the stopping code) from successive epochs falling below $10^{-4}$. The report plots training loss and average training error against epochs for the explored configurations. Training error is defined as 1 − training accuracy.

### 6.1 Average training error

For an epoch with training accuracy A, the average training error rate is $E = 1 - A$. For example, training accuracy of 0.95 corresponds to a training error rate of 0.05. A decreasing error curve indicates that the model is learning the training examples; comparison with validation and test accuracy helps identify generalization gaps.

## 7. Baseline Hyperparameter Experiment Results

The notebook's recorded baseline experiment table is summarized below. Values are taken from the stored results output.

| Model | Hidden units | Layers | Epochs | Train acc. | Validation acc. | Test acc. | Test loss |
|---|---|---|---|---|---|---|---|
| **RNN_h64_layers1** | 64 | 1 | 83 | 99.43% | 97.17% | 91.36% | 0.5059 |
| **RNN_h128_layers2** | 128 | 2 | 45 | 99.95% | 98.89% | 93.00% | 0.4117 |
| **LSTM_h64_layers1** | 64 | 1 | 30 | 99.98% | 98.89% | 96.45% | 0.1670 |
| **LSTM_h128_layers2** | 128 | 2 | 20 | 99.95% | 99.70% | 96.64% | 0.1634 |
| **LSTM_h256_layers2** | 256 | 2 | 17 | 100.00% | 99.60% | 96.41% | 0.1918 |
| **GRU_h64_layers1** | 64 | 1 | 34 | 99.98% | 99.19% | 95.73% | 0.2230 |
| **GRU_h128_layers2** | 128 | 2 | 26 | 99.98% | 99.60% | 95.91% | 0.1928 |
| **GRU_h256_layers2** | 256 | 2 | 10 | 99.13% | 98.79% | 95.68% | 0.1604 |

Among the selected baseline configurations, LSTM_h128_layers2 had the highest test accuracy at 96.64%. It used 128 hidden units, two recurrent layers, dropout 0.3 and learning rate 0.001. Its validation accuracy was 99.70%, and it trained for 20 epochs according to the recorded results.

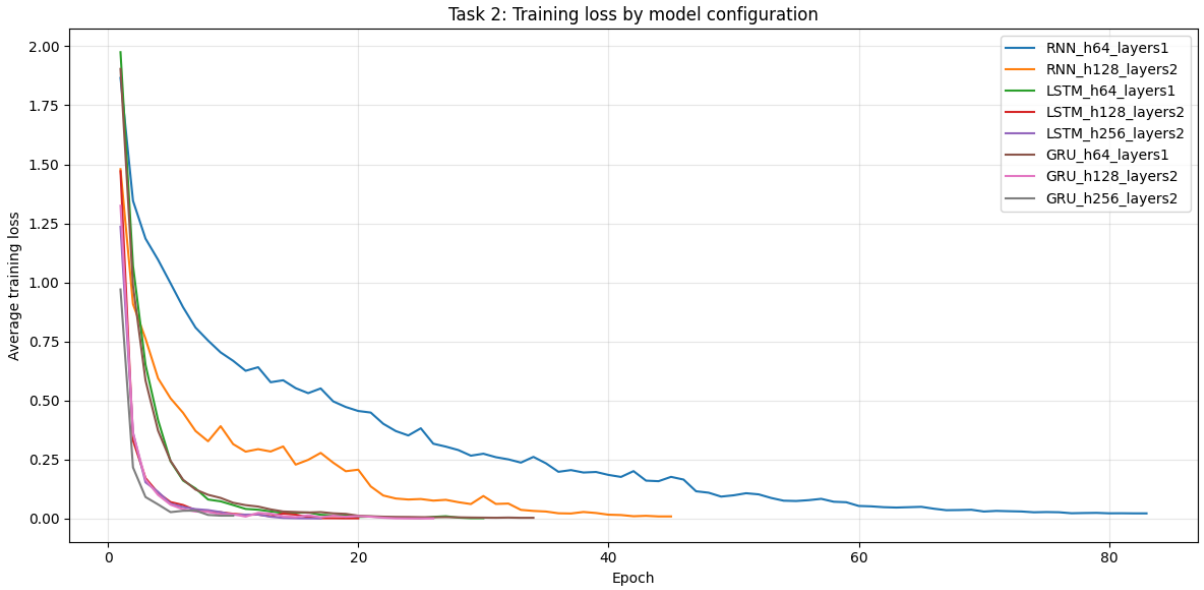



*Figure 1. Training loss versus epoch for baseline hyperparameter configurations.*

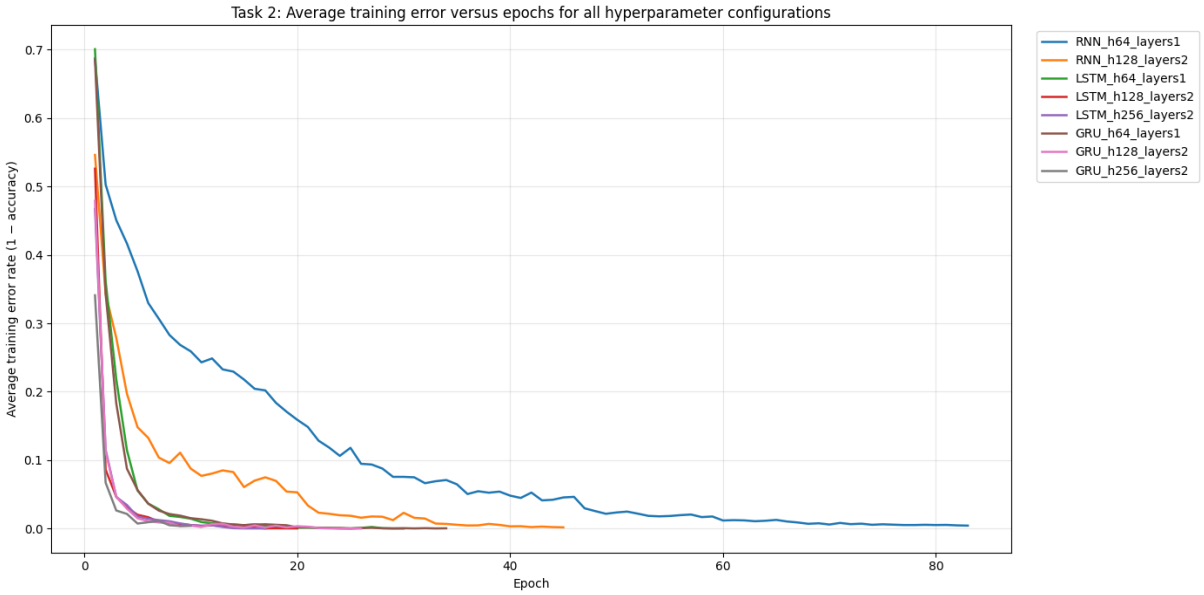



*Figure 2. Average training error (1 − training accuracy) versus epoch for baseline configurations.*

## 8. Baseline Model Evaluation

| Architecture (best baseline) | Selected configuration | Validation accuracy | Training accuracy | Test accuracy |
|---|---|---|---|---|
| **RNN** | 128 hidden units, 2 layers, dropout 0.3 | 98.89% | 99.95% | 93.00% |
| **LSTM** | 128 hidden units, 2 layers, dropout 0.3 | 99.70% | 99.95% | 96.64% |

| | | | | |
|---|---|---|---|---|
| **GRU** | 128 hidden units, 2 layers, dropout 0.3 | 99.60% | 99.98% | 95.91% |

The LSTM baseline outperformed the RNN and GRU baselines on the held-out test set. The Vanilla RNN reached 93.00%, while the GRU reached 95.91% and the LSTM reached 96.64%. These results suggest that gated recurrent cells are better able to represent the temporal patterns in these MFCC sequences than the baseline Vanilla RNN in this experiment.

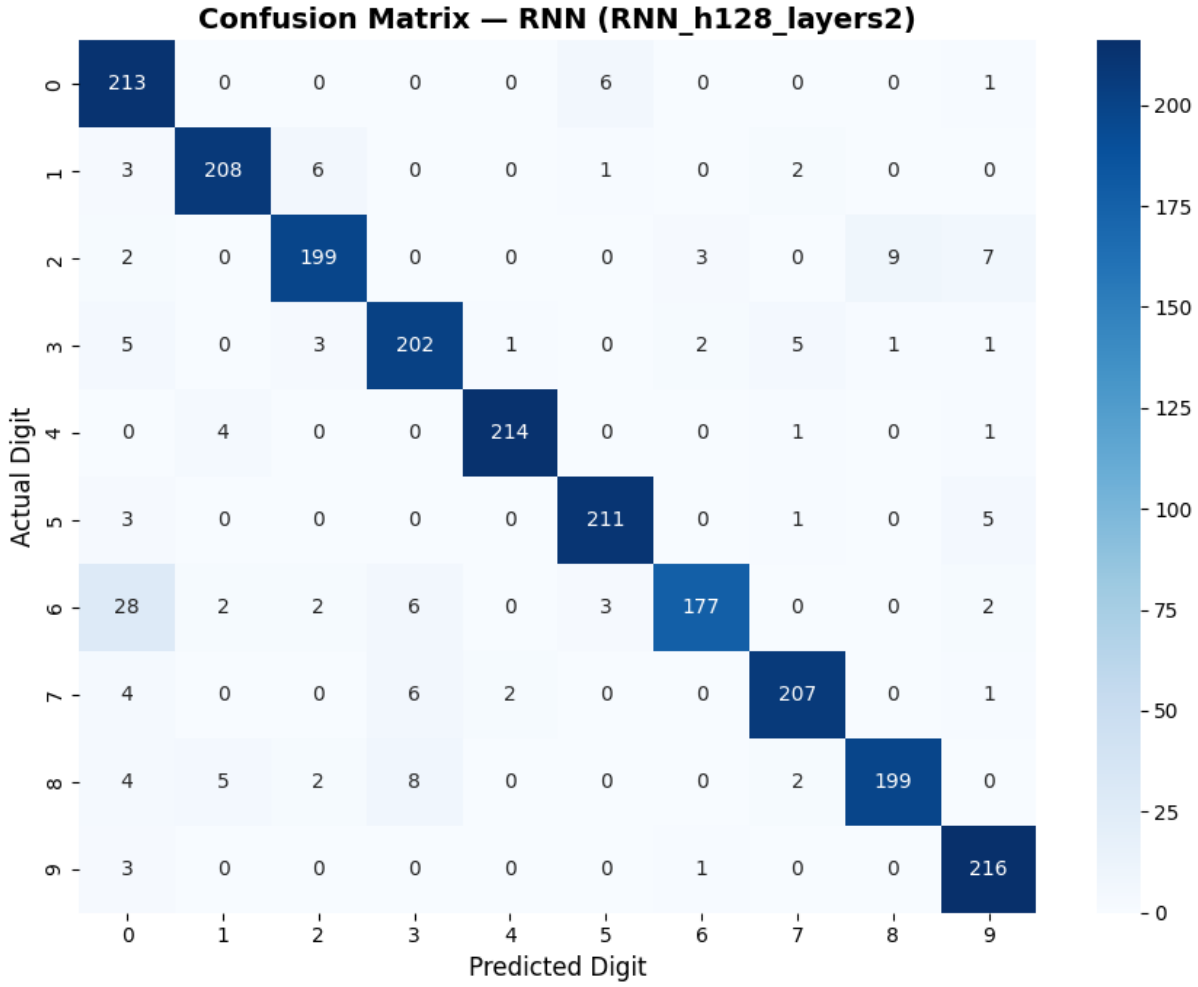



*Figure 3. Confusion matrix for the selected baseline RNN.*

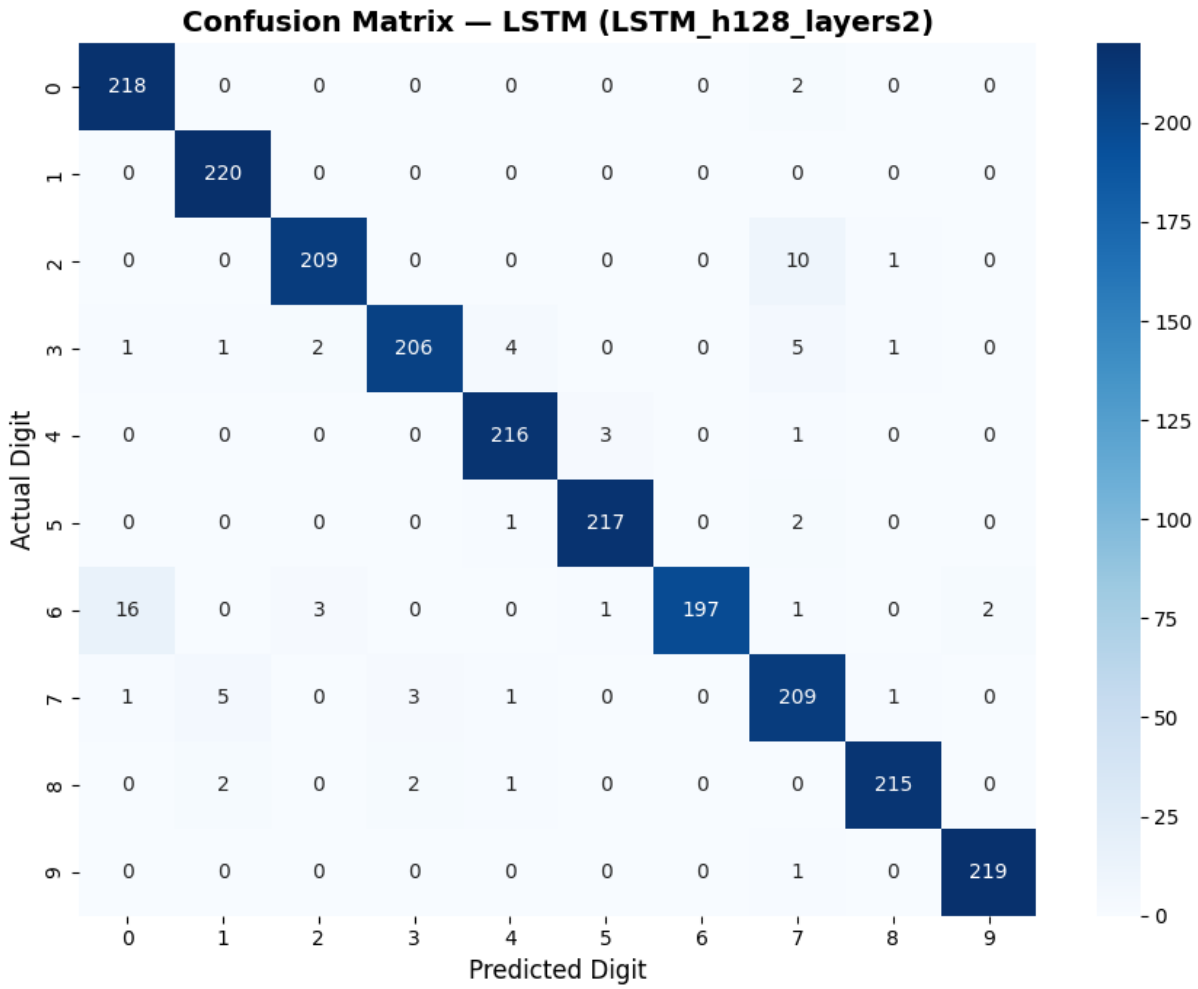



*Figure 4. Confusion matrix for the selected baseline LSTM.*

## 9. Bonus Experiments: Bidirectionality and Attention

The notebook evaluates 12 advanced configurations covering standard recurrent models, bidirectional models, attention-pooling models and bidirectional models with attention. This comparison isolates the contribution of processing sequences in both directions and applying learned temporal weights.

| Configuration | Train accuracy | Validation accuracy | Test accuracy | Test loss |
|---|---|---|---|---|
| **Vanilla RNN** | 94.40% | 94.65% | 90.73% | 0.3846 |
| **LSTM** | 99.54% | 99.09% | 95.86% | 0.1739 |
| **GRU** | 99.95% | 99.49% | 96.45% | 0.1557 |
| **Bi-RNN** | 98.81% | 98.48% | 95.95% | 0.1702 |
| **Bi-LSTM** | 99.89% | 99.49% | 97.55% | 0.1035 |
| **Bi-GRU** | 99.91% | 99.49% | 98.23% | 0.0649 |
| **RNN + Attention** | 98.40% | 97.17% | 93.59% | 0.2476 |
| **LSTM + Attention** | 99.79% | 98.89% | 97.77% | 0.0976 |
| **GRU + Attention** | 99.50% | 99.29% | 97.14% | 0.1015 |
| **Bi-RNN + Attention** | 99.52% | 98.99% | 97.45% | 0.1143 |
| **Bi-LSTM + Attention** | 99.98% | 99.49% | 98.27% | 0.0607 |
| **Bi-GRU +** | 100.00% | 99.60% | 98.73% | 0.0672 |

## Attention (Bonus)

The best advanced model selected by validation accuracy was Bi-GRU_Attention. Its validation accuracy was 99.60%, and its test accuracy was 98.73%. The corresponding training accuracy was 100.00%, with test loss 0.0672. This is the strongest test result among the 12 advanced configurations reported in the notebook.

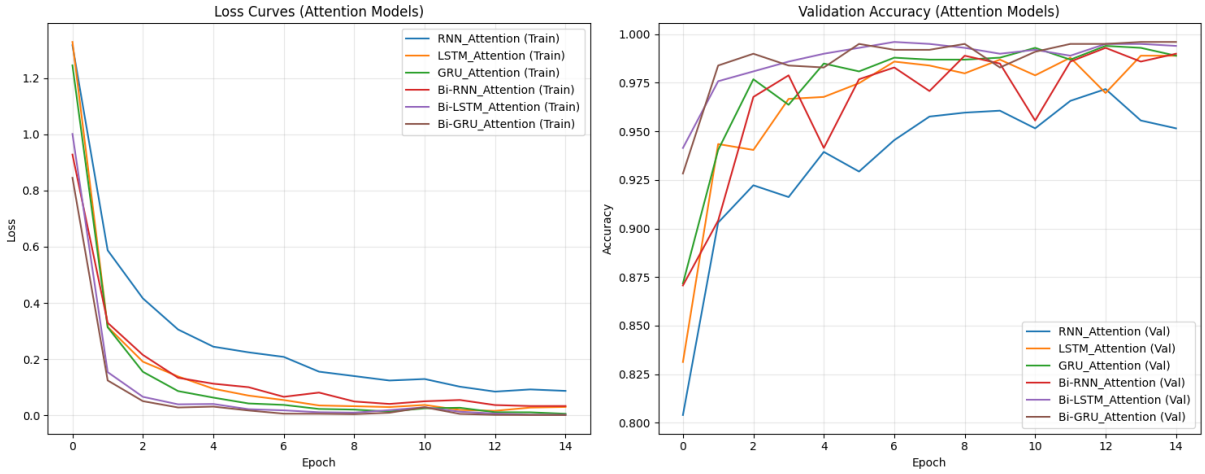



*Figure 5. Loss and accuracy curves for attention-based advanced models.*

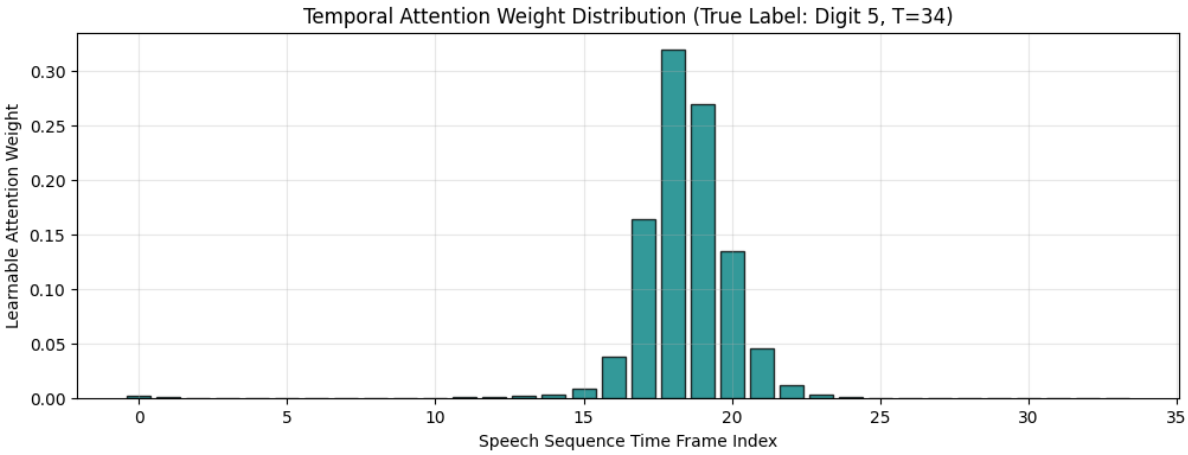



*Figure 6. Temporal attention weights for a representative test utterance of digit 5.*

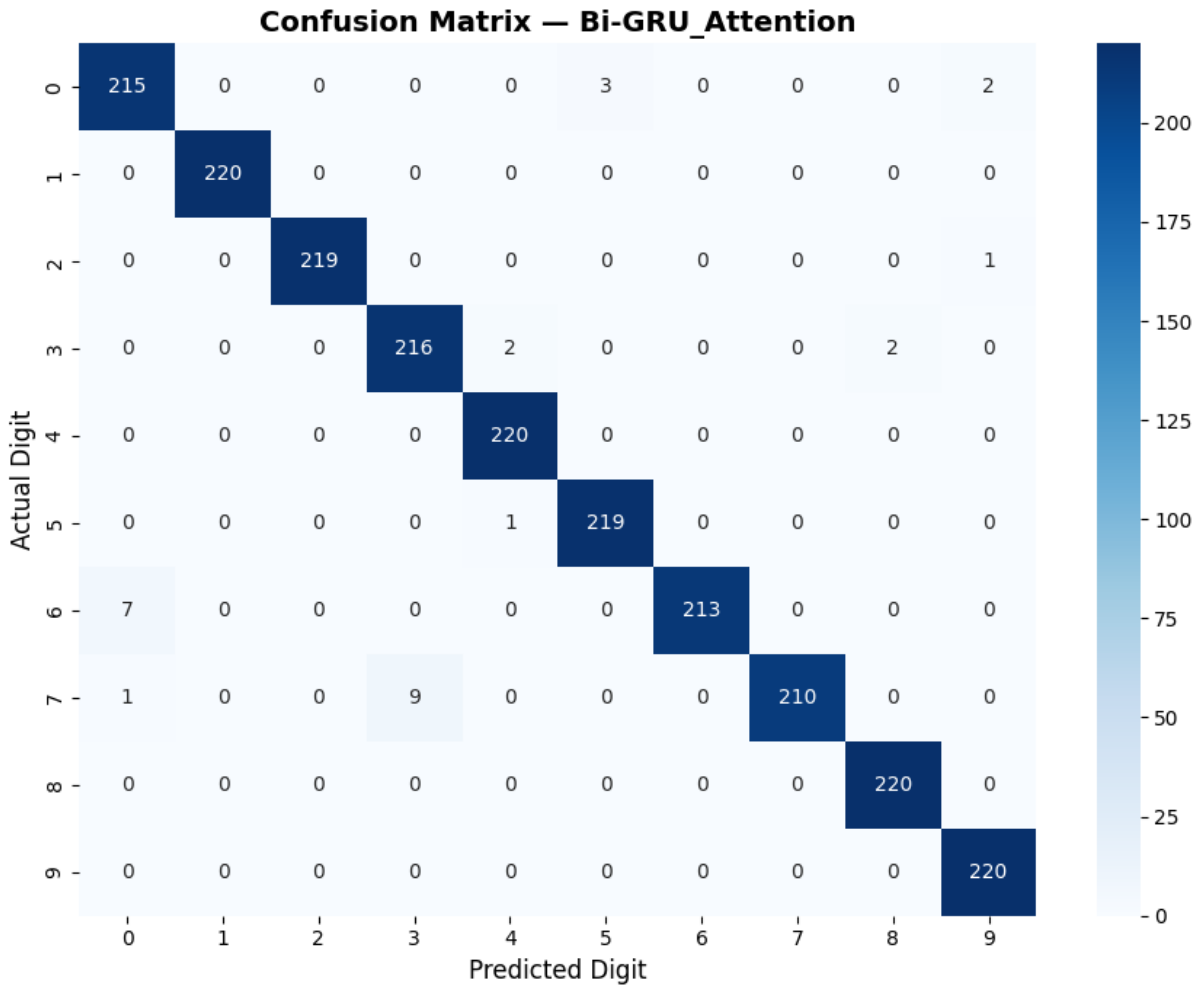



*Figure 7. Confusion matrix for the best advanced model (Bi-GRU with temporal attention).*

## 10. Comparison and Discussion

- Gated recurrent cells improve over the Vanilla RNN in the recorded comparisons: the baseline LSTM and GRU both achieved higher test accuracy than the baseline RNN.
- Bidirectional processing improves performance in the advanced experiments. Bi-GRU without attention achieved 98.23% test accuracy, compared with 96.45% for the unidirectional GRU in that same comparison.
- Temporal attention gives further gains when paired with bidirectional models: Bi-LSTM + Attention reached 98.27%, and Bi-GRU + Attention reached 98.73%.
- In this recorded advanced comparison, adding attention improves test accuracy over the corresponding non-attention model for each recurrent family. The strongest result is obtained by combining bidirectional processing with attention: Bi-GRU + Attention reaches 98.73%, above Bi-GRU without attention at 98.23%.
- Training accuracies near 100% and slightly lower test accuracies indicate a generalization gap. This is expected when models fit the training data very well; the held-out test performance remains the key comparison.

Important distinction: the notebook contains two sets of experiments: the baseline hyperparameter search and the advanced 12-configuration bonus experiment. Their values come from separate training runs and should not be mixed as though they were one experiment. The best baseline test result was 96.64% (LSTM_h128_layers2), while the best advanced result was 98.73% (Bi-GRU_Attention).

## 11. Confusion Matrix and Classification Metrics

A confusion matrix compares actual digit labels (rows) against predicted labels (columns). Correct predictions lie on the main diagonal, while off-diagonal values identify errors between digits. The notebook produces confusion matrices for the best RNN, LSTM and GRU baseline models and saves a confusion matrix for the best advanced model.

The classification reports show precision, recall and F1-score for each digit. Precision measures the fraction of predictions for a digit that were correct; recall measures the fraction of actual examples of that digit that were recovered; F1-score is the harmonic mean of precision and recall. In the baseline results, the LSTM macro-average F1-score was 0.9663, compared with 0.9301 for the RNN and 0.9593 for the GRU.

## 12. MFCC Feature Visualization

The notebook creates MFCC heatmaps for representative utterances from the ten digit classes. In these heatmaps, the horizontal axis corresponds to time frames, the vertical axis corresponds to the 13 MFCC coefficients, and color intensity represents the coefficient value. The plots allow qualitative inspection of how acoustic patterns evolve through an utterance and differ between spoken digits.

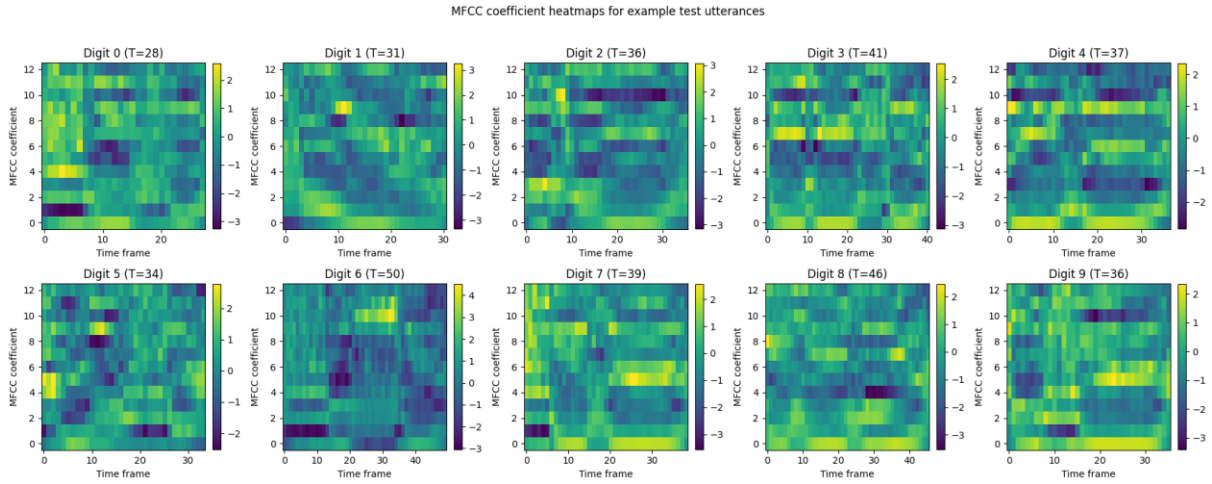



*Figure 8. MFCC coefficient heatmaps for representative spoken digit utterances.*

## 13. Conclusion

This task implemented sequential spoken-digit classification using pre-computed 13-dimensional MFCC sequences. The notebook handled variable-length utterances using padded mini-batches and trained Vanilla RNN, LSTM and GRU models. The baseline hyperparameter comparison showed that the selected two-layer LSTM achieved the best baseline test accuracy of 96.64%. The bonus experiments explored bidirectional processing and temporal attention, with the Bi-GRU plus attention model reaching 99.60% validation accuracy and 98.73% test accuracy. Training-error curves, classification reports, confusion matrices, attention weights and MFCC heatmaps provide complementary views of learning behavior and model performance. Overall, the experiments indicate that modeling both directions of a sequence and weighting informative timesteps can improve classification of these MFCC utterances.